

Look, Ma, No Matrices!

Steven De Keninck
steven@enki.ws
University of Amsterdam
Amsterdam, Netherlands



Figure 1: Look, Ma, No Matrices!

ABSTRACT

This talk discusses some of the pitfalls and surprises revealed by the "Look, Ma, No Matrices!" [Keninck 2024] project that implements an industry standard forward renderer exclusively using PGA (Projective or Plane-based Geometric Algebra). We will briefly go over some of the challenges and reveal how and insightful application of PGA leads to substantial savings for a typical tangent space normal-mapping setup.

CCS CONCEPTS

• Computing methodologies → Computer graphics.

KEYWORDS

Projective Geometric Algebra, Realtime Rendering, Tangent space normalmapping, Dual Quaternion Skinning

ACM Reference Format:

Steven De Keninck. 2024. Look, Ma, No Matrices!. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH Talks '24)*, July 27–August 01, 2024, Denver, CO, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3641233.3665801>

1 INTRODUCTION

Since the 2019 SIGGRAPH course [Gunn and De Keninck 2019], Geometric Algebra, and the Euclidean PGA in particular, has been

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH Talks '24, July 27–August 01, 2024, Denver, CO, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0515-1/24/07

<https://doi.org/10.1145/3641233.3665801>

gaining traction within the computer graphics and machine learning communities [Bosch 2020; Brehmer et al. 2023; Ruhe et al. 2023]. Despite its broad applicability, including for higher dimensional geometry and physics, its adoption in traditional 3D graphics has been limited, often merely rebranding a dual quaternion as a PGA motor. The 'Look, Ma, No Matrices!' project [Keninck 2024] aims to broaden PGA's application by introducing a modern, glTF-compliant, forward-rendering 3D engine that fully integrates the PGA algebra.

In this presentation, we reveal surprising results from our project that question the prevailing belief in the unparalleled efficiency of matrix-based approaches. We begin with a brief overview of PGA before demonstrating how its application in vertex descriptor and motor-based transformations offers a compelling alternative to traditional matrix methods. This novel approach not only competes on performance, even with static meshes, but also achieves significant memory savings (25% in vertex descriptor, 75% on transform uniforms). The resulting code snippet can be substituted directly in an existing pipeline.

2 TEASER RESULTS

Tangent space normalmapping is a widely used technique, that requires the construction of an orthogonal 3x3 matrix (called the TBN matrix), which is used to transform a sampled normal into the world frame for lighting calculations. The computational expense here typically involves transforming object space normal and tangent vectors to worldspace, so they can be interpolated for the fragment shader. It is this computation that we are revisiting.

To recap, the traditional 4x4 matrix vector product involves 16 multiplications and 12 additions, whereas the optimal PGA motor (dual quaternion) requires 21 multiplications and 18 additions. So,

one would assume that transforming the position, normal, and tangent of a normal-mapped mesh incurs this computational expense threefold, seemingly an inescapable overhead. Yet, the proposed presentation reveals how a consistent application of PGA principles leads to some unexpected improvements:

Table 1: Comparison of vertex descriptor size, uniform size and computational cost in the vertex shader. Both scenarios use an identical fragment shader.

	Khronos glTF	PGA	Difference
Vertex Descriptor	3+3+4+2 = 12	3+4+2 = 9	-3 (-25%)
Uniforms	32	8	-24 (-75%)
Multiplications	48	47	-1
Additions	36	38	+2

As table 1 shows, we can get substantial savings on the size of our vertex descriptor for a virtually identical computational cost in the vertex shader.

A detailed write-up and complete source code is available at the project page, we limit ourselves here to some extracts showing efficient rotor composition and how normal and tangent vectors can efficiently be extracted from the quaternion part of a PGA motor.

```
#define motor      mat2x4    // [ [ s, e23, e31, e12], [e01, e02, e03, e0123] ]
#define point      vec3      // [ e032, e013, e021 ] with implied 1 e123
#define direction  vec3      // [ e032, e013, e021 ] with implied 0 e123

// Geometric product of two origin preserving rotations.
// 16 mul, 12 add
motor gp_rr( motor a, motor b ) {
    return motor(
        a[0].x*b[0] + vec4(-dot(a[0].yzw, b[0].yzw), b[0].x*a[0].yzw + cross(b[0].yzw, a[0].yzw)),
        vec4(0.)
    );
}

// Extract the (half size) z and x directions from a normalized motor.
// 9 mul, 8 add
void extractNormalTangent( motor a, out direction normal, out direction tangent ) {
    float yw = a[0].y * a[0].w;
    float xz = a[0].x * a[0].z;
    float zz = a[0].z * a[0].z;

    normal = direction( yw - xz, a[0].z*a[0].w + a[0].y*a[0].x, 0.5 - zz - a[0].y*a[0].y );
    tangent = direction( 0.5 - zz - a[0].w*a[0].w, a[0].z*a[0].y - a[0].x*a[0].w, yw + xz );
}
```

3 TALK OVERVIEW

(1) 5m - PGA: An algebra of geometry.

A minimal introduction to outline the PGA concepts used in the talk. Introducing and explaining the reasons for a two-sided product, the origin of the half-angle, and what a double cover really is.

(2) 5m - Motors and lean sandwiches.

Showing how PGA motors include and extend the quaternions and dual quaternions, and how to optimize the various products involved. This includes a short overview of the actual code (glsl), going into details specifically on composition and sandwich operators.

(3) 5m - Don't be normal, or tangent.

We discuss how a tangentRotor can replace current normal and tangent vertex attributes, and how this can be done without introducing any extra cost in the vertex shader.

(4) 5m - Conclusions and Results.

We show some of the example implementation and its vertex shaders, and go over some of the performance measurements.

REFERENCES

- Marc Ten Bosch. 2020. N-dimensional rigid body dynamics. *ACM Trans. Graph.* 39, 4, Article 55 (aug 2020), 6 pages. <https://doi.org/10.1145/3386569.3392483>
- Johann Brehmer, Pim de Haan, Sönke Behrends, and Taco Cohen. 2023. Geometric Algebra Transformer. *arXiv:2305.18415* [cs.LG]
- Charles G. Gunn and Steven De Keninck. 2019. Geometric Algebra and Computer Graphics. In *ACM SIGGRAPH 2019 Courses* (Los Angeles, California) (*SIGGRAPH '19*). Association for Computing Machinery, New York, NY, USA, Article 12, 140 pages. <https://doi.org/10.1145/3305366.3328099>
- Steven De Keninck. 2024. Look, Ma, No Matrices! <https://enkimute.github.io/LookMaNoMatrices/>.
- David Ruhe, Jayesh K. Gupta, Steven de Keninck, Max Welling, and Johannes Brandstetter. 2023. Geometric Clifford Algebra Networks. *arXiv:2302.06594* [cs.LG]